

CO4 AT2-CASE STUDY

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: DSA0108

CASE STUDY SCENARIO

A multi-specialty hospital is planning to modernize its Hospital Management System (HMS) to efficiently manage different types of patients such as Inpatients, Outpatients, and Emergency Patients.

The current system has several issues:

- Redundant data storage for patient details
- Lack of flexibility in handling different billing mechanisms
- Difficulty in extending the system for new patient categories
- Inefficient handling of patient records in memory

To improve system performance and scalability, the hospital wants to redesign the system using advanced Object-Oriented Programming concepts in C++.

he system using advanced Object-Oriented Programming concepts in C++.

Case Study Task

You are required to analyze, design, implement, and evaluate a C++ application using:

- Inheritance (types of inheritance)
- Virtual base classes
- Abstract classes
- Constructors in derived classes
- Member classes (composition)
- Pointers and pointers to objects
- this pointer
- Pointer to derived classes
- Runtime polymorphism

Part A:

1. Identify the limitations of the existing system.
2. Explain how OOP principles can solve these issues.
3. List the functional requirements of the improved system.

Part B:

4. Design a class hierarchy:
 - o Abstract base class: Patient
 - o Derived classes:
 - ♣ InPatient
 - ♣ OutPatient
 - ♣ EmergencyPatient
5. Extend using multiple inheritance:
 - o Create CriticalPatient (inherits from InPatient and EmergencyPatient)
6. Use:
 - o Virtual base classes to resolve ambiguity
 - o A member class (Doctor) inside Patient

Part C:

7. Compare:
 - o Inheritance vs Composition in patient management
 - o Static vs Dynamic binding
8. Analyze:
 - o The role of virtual base classes in diamond problem
 - o Benefits of pointers in dynamic patient handling
9. Justify:
 - o Your design approach
 - o How it improves scalability and maintainability

Part D:

Develop a C++ program that:

10. Uses:
 - o Abstract class with pure virtual function calculateBill()
 - o Function overriding (polymorphism)
11. Implements billing logic:
 - o InPatient → Room charge + Treatment cost
 - o OutPatient → Consultation fee
 - o Emergency → Additional emergency charges
12. Demonstrates:
 - o Pointer to base class (Patient*)
 - o Dynamic allocation (new, delete)
 - o Use of this pointer
13. Includes:
 - o Menu-driven program using control structures

Sample Scenario

Input:

Patient Type: Critical Patient

Room Charge: 2000

Treatment Cost: 3000

Emergency Charge: 1500

Output:

Total Bill = ₹6500

Code:

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <limits>
5 using namespace std;
6 // Part B: Abstract base class Patient
7 class Patient {
8 protected:
9     string name;
10    int id;
11
12    // Part B: Member class (Composition)
13    class Doctor {
14 public:
15        string doctorName;
16        void setDoctor(string dName) { doctorName = dName; }
17    };
18
19    Doctor assignedDoctor;
20
21 public:
22     Patient(string n, int i, string dName) : name(n), id(i) {
23         assignedDoctor.setDoctor(dName);
24     }
25
26     // Part D: Pure virtual function for polymorphism
27     virtual void calculateBill() = 0;
28
29     virtual void displayDetails() {
30         cout << "\nPatient ID: " << this->id << endl;
31         cout << "Name: " << this->name << endl;
32         cout << "Doctor: " << this->assignedDoctor.doctorName << endl;
33     }
34
35     virtual ~Patient() {} // Virtual destructor for safe cleanup
36 };
37
38 // Part B: Derived class InPatient (Virtual base class for diamond problem)
39 class InPatient : virtual public Patient {
```

1
John
101
Dr. Smith
500
2500
4

```

40 protected:
41     double roomCharge;
42     double treatmentCost;
43
44 public:
45     InPatient(string n, int i, string d, double rc, double tc)
46         : Patient(n, i, d), roomCharge(rc), treatmentCost(tc) {}
47
48     void calculateBill() override {
49         cout << "Total Bill (InPatient) = " << (roomCharge + treatmentCost) << endl;
50     }
51 };
52
53 // Part B: Derived class OutPatient
54 class OutPatient : public Patient {
55 private:
56     double consultationFee;
57
58 public:
59     OutPatient(string n, int i, string d, double cf)
60         : Patient(n, i, d), consultationFee(cf) {}
61
62     void calculateBill() override {
63         cout << "Total Bill (OutPatient) = " << consultationFee << endl;
64     }
65 };
66
67 // Part B: Derived class EmergencyPatient (Virtual base class for diamond problem)
68 class EmergencyPatient : virtual public Patient {
69 protected:
70     double emergencyCharge;
71
72 public:
73     EmergencyPatient(string n, int i, string d, double ec)
74         : Patient(n, i, d), emergencyCharge(ec) {}
75
76     void calculateBill() override {
77         cout << "Total Bill (Emergency) = " << emergencyCharge << endl;

```

```

78     }
79 };
80
81 // Part B: Multiple Inheritance - CriticalPatient
82 class CriticalPatient : public InPatient, public EmergencyPatient {
83 private:
84     double extraEmergencyCharge;
85
86 public:
87     CriticalPatient(string n, int i, string d, double rc, double tc, double ec, double ec)
88         : Patient(n, i, d), InPatient(n, i, d, rc, tc), EmergencyPatient(n, i, d, ec)
89
90     void calculateBill() override {
91         // Logic: Room + Treatment + Emergency + Extra
92         double total = roomCharge + treatmentCost + emergencyCharge + extraEmergencyCharge;
93         cout << "Total Bill (Critical Patient) = " << total << endl;
94     }
95 };
96
97 int main() {
98     int choice;
99     Patient* ptr = nullptr;
100
101     while (true) {
102         cout << "\n--- Hospital Management System ---" << endl;
103         cout << "1. InPatient" << endl;
104         cout << "2. OutPatient" << endl;
105         cout << "3. Critical Patient" << endl;
106         cout << "4. Exit" << endl;
107         cout << "Enter choice: ";
108
109         if (!(cin >> choice)) {
110             cout << "Invalid input. Please enter a number." << endl;
111             cin.clear();
112             cin.ignore(numeric_limits<streamsize>::max(), '\n');
113             continue;
114         }
115

```

```

116     if (choice == 4) break;
117
118     if (choice >= 1 && choice <= 3) {
119         string name, doc;
120         int id;
121
122         cout << "Enter Name: ";
123         cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Clear buffer
124         getline(cin, name);
125
126         cout << "Enter ID: ";
127         cin >> id;
128
129         cout << "Enter Doctor Name: ";
130         cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Clear buffer
131         getline(cin, doc);
132
133         if (choice == 1) {
134             double rc, tc;
135
136             cout << "Enter Room Charge: "; cin >> rc;
137             cout << "Enter Treatment Cost: "; cin >> tc;
138             ptr = new InPatient(name, id, doc, rc, tc);
139         }
140         else if (choice == 2) {
141             double cf;
142             cout << "Enter Consultation Fee: "; cin >> cf;
143             ptr = new OutPatient(name, id, doc, cf);
144         }
145         else if (choice == 3) {
146             double rc, tc, ec, extra;
147             cout << "Enter Room Charge: "; cin >> rc;
148             cout << "Enter Treatment Cost: "; cin >> tc;
149             cout << "Enter Emergency Charge: "; cin >> ec;
150             cout << "Enter Additional Emergency Charge: "; cin >> extra;
151             ptr = new CriticalPatient(name, id, doc, rc, tc, ec, extra);
152         }
153
154         if (ptr != nullptr) {

```

```

154         ptr->displayDetails();
155         ptr->calculateBill();
156         delete ptr;
157         ptr = nullptr;
158     }
159     } else {
160         cout << "Invalid choice. Please try again." << endl;
161     }
162 }
163
164 cout << "Exiting System..." << endl;
165 return 0;
166 }

```

```

#include <iostream>
#include <string>
#include <vector>
#include <limits>

using namespace std;

// Part B: Abstract base class Patient
class Patient {
protected:
    string name;
    int id;

// Part B: Member class (Composition)
    class Doctor {
    public:
        string doctorName;
        void setDoctor(string dName) { doctorName = dName; }
    };

    Doctor assignedDoctor;

    public:
    Patient(string n, int i, string dName) : name(n), id(i) {
        assignedDoctor.setDoctor(dName);
    }

// Part D: Pure virtual function for polymorphism
    virtual void calculateBill() = 0;

    virtual void displayDetails() {
        cout << "\nPatient ID: " << this->id << endl;
        cout << "Name: " << this->name << endl;
        cout << "Doctor: " << this->assignedDoctor.doctorName << endl;
    }

    virtual ~Patient() {} // Virtual destructor for safe cleanup
};

```

```

// Part B: Derived class InPatient (Virtual base class for diamond problem)
class InPatient : virtual public Patient {
protected:
double roomCharge;
double treatmentCost;

public:
InPatient(string n, int i, string d, double rc, double tc)
: Patient(n, i, d), roomCharge(rc), treatmentCost(tc) {}

void calculateBill() override {
cout << "Total Bill (InPatient) = " << (roomCharge + treatmentCost) << endl;
}
};

// Part B: Derived class OutPatient
class OutPatient : public Patient {
private:
double consultationFee;

public:
OutPatient(string n, int i, string d, double cf)
: Patient(n, i, d), consultationFee(cf) {}

void calculateBill() override {
cout << "Total Bill (OutPatient) = " << consultationFee << endl;
}
};

// Part B: Derived class EmergencyPatient (Virtual base class for diamond
problem)
class EmergencyPatient : virtual public Patient {
protected:
double emergencyCharge;

public:
EmergencyPatient(string n, int i, string d, double ec)
: Patient(n, i, d), emergencyCharge(ec) {}

void calculateBill() override {
cout << "Total Bill (Emergency) = " << emergencyCharge << endl;
}
};

// Part B: Multiple Inheritance - CriticalPatient
class CriticalPatient : public InPatient, public EmergencyPatient {
private:
double extraEmergencyCharge;

public:
CriticalPatient(string n, int i, string d, double rc, double tc, double ec,
double extra)
: Patient(n, i, d), InPatient(n, i, d, rc, tc), EmergencyPatient(n, i, d,
ec), extraEmergencyCharge(extra) {}

void calculateBill() override {
// Logic: Room + Treatment + Emergency + Extra

```

```

double total = roomCharge + treatmentCost + emergencyCharge +
extraEmergencyCharge;
cout << "Total Bill (Critical Patient) = " << total << endl;
}
};

int main() {
int choice;
Patient* ptr = nullptr;

while (true) {
cout << "\n--- Hospital Management System ---" << endl;
cout << "1. InPatient" << endl;
cout << "2. OutPatient" << endl;
cout << "3. Critical Patient" << endl;
cout << "4. Exit" << endl;
cout << "Enter choice: ";

if (!(cin >> choice)) {
cout << "Invalid input. Please enter a number." << endl;
cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
continue;
}

if (choice == 4) break;

if (choice >= 1 && choice <= 3) {
string name, doc;
int id;

cout << "Enter Name: ";
cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Clear buffer
getline(cin, name);

cout << "Enter ID: ";
cin >> id;

cout << "Enter Doctor Name: ";
cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Clear buffer
getline(cin, doc);

if (choice == 1) {
double rc, tc;
cout << "Enter Room Charge: "; cin >> rc;
cout << "Enter Treatment Cost: "; cin >> tc;
ptr = new InPatient(name, id, doc, rc, tc);
}
else if (choice == 2) {
double cf;
cout << "Enter Consultation Fee: "; cin >> cf;
ptr = new OutPatient(name, id, doc, cf);
}
else if (choice == 3) {
double rc, tc, ec, extra;
cout << "Enter Room Charge: "; cin >> rc;
cout << "Enter Treatment Cost: "; cin >> tc;

```

```

cout << "Enter Emergency Charge: "; cin >> ec;
cout << "Enter Additional Emergency Charge: "; cin >> extra;
ptr = new CriticalPatient(name, id, doc, rc, tc, ec, extra);
}

if (ptr != nullptr) {
ptr->displayDetails();
ptr->calculateBill();
delete ptr;
ptr = nullptr;
} else {
cout << "Invalid choice. Please try again." << endl;
}
}

cout << "Exiting System..." << endl;
return 0;
}

```

Output:

```

--- Hospital Management System ---
1. InPatient
2. OutPatient
3. Critical Patient
4. Exit
Enter choice: Enter Name: Enter ID: Enter Doctor Name: Enter Room Charge: Enter
Treatment Cost:
Patient ID: 101
Name: John
Doctor: Dr. Smith
Total Bill (InPatient) = 3000
--- Hospital Management System ---
1. InPatient
2. OutPatient
3. Critical Patient
4. Exit
Enter choice: Exiting System...

```